# ECE 7670
# Program 5
# Nathan Walker

October 3, 2025

---

1. Write down the vector, polynomial, and power representations of the field GF(23) generated with the polynomial g(x)=1 + x +x3. Based on this, write down the tables v2p and p2v for this field. (See the implementation suggestions for the definition of these tables.)

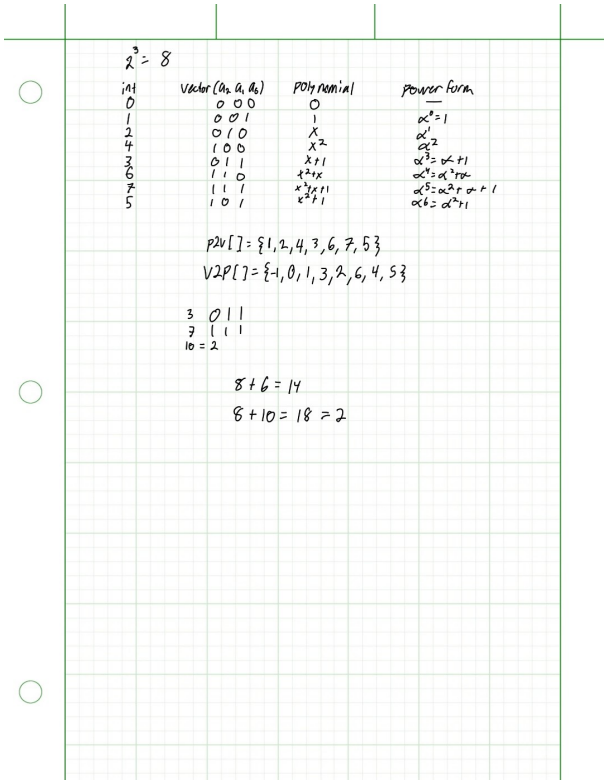



Figure 1: Hand work of the preliminary exercises

2. Create a C++ class GFNUM2m with overloaded operators to implement arithmetic over the field GF(2m) for an arbitrary m¡32. This is similar instructure to classModAr, except that the details of the arithmetic are different. Test all operations of your class:+,-,*,/,+,+=,-=, *=,/=,+=+,==, !=for the field generated by g(x)=1+x+x4 by comparing the results the computer provides with results you

calculate by hand. Then test for GF(23) generated by g(x)=1+x+x3.

3. write your code, then extensively test the arithmetic using GF(24) (as shown above) and GF(28). For the field GF(28), use the primitive polynomial p(x)= 1 +x2 +x3 +x4 +x8: GFNUM2m::initgf(8,0x11D); // 1 0001 1101 // x8+x4+x3+x2+1

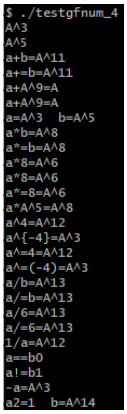



Figure 2: output of GF2$^4$*example*

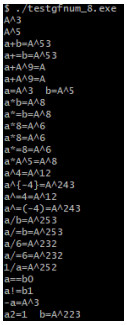



Figure 3: output of GF2$^8$*example*

```
// GFNUM2m.h -- GF(2^m) numbers
// Created by Todd K. Moon
// Copyright 2004 by Todd K. Moon
// Permission is granted to use this program/data
// for educational/research only

#ifndef GFNUM2m_H
#define GFNUM2m_H
#include <iostream>
using namespace std;

#include "GF2.h"

enum outformat{GFvector,GFpower};  // output options for GFNUM2m
class GFNUM2m;                     // declare the class (for the
   externs below)
```

```
16
17 extern GFNUM2m ALPHA;           // set up a global alpha
18 extern GFNUM2m& A;              // and a reference to alpha for
       shorthand
19
20 class GFNUM2m {
21 protected:
22    int v;                       // the value of the element (in
          vector form)
23
24    static int gfm;              // number of elements in vector
          representation
25    static int gfN;              // number of nonzero elments in
          field
26    static outformat outtype;    // output type (vector or power
          representation)
27    static int *v2p;             // convert from vector to power
          notation
28                                 // (there is no exponential notation
                                       for zero)
29    static int *p2v;             // convert from power to vector
          representation
30    static void gferror(const char *s) {
31       cerr << s;
32    } // print an error message
33 public:
34    // static functions
35    static void initgf(int m); // initialize multiplication array
36    static void initgf(int m, unsigned int g); // initialize mult.
          array
37    static void cleargf() {      // get rid of previous tables
38       delete [] p2v;
39       delete [] v2p;
40       p2v = 0;  v2p = 0;
41    }
42    static void setouttype(outformat ot) { outtype = ot;} // set the
          output type
43
44    // Constructors
45    GFNUM2m(const GFNUM2m& newnum) { v = newnum.v;}  // assignment
46    GFNUM2m(const int newv) { v = newv;};
47                                  // pass in a number in vector
                                         representation
48    GFNUM2m(void) { v = 0;};      // default constructor 0 element)
49    GFNUM2m(const GF2 gf) { v = gf.getv();};
50    ~GFNUM2m() {};               // destructor
51
52    // Functions to get member information
53    int character() const { return 2; };
54    int getv(void) const { return v; };
55                                 // return the vector representation
```

```
56    int getp(void) const { return v2p[v]; };
57                                  // return the power representation
58    static int getN(void) { return gfN; };
59                                  // return N, number of nonzero elts.
60    static int getm(void) { return gfm; };
61                                  // return m (degree of vector sp.)
62    static outformat getouttype() { return outtype; };
63                                  // return the output type
64
65    //
          ***********************************************************************
66    //                      the overloaded operators
67    //
          ***********************************************************************
68
69    GFNUM2m& operator+=(const GFNUM2m &a) { v ^= a.v;  return *this;
          }
70    GFNUM2m& operator-=(const GFNUM2m &a) { v ^= a.v;  return *this;
          }
71    GFNUM2m& operator*=(const GFNUM2m &a) {
72       if(a.v == 0 || v==0) v = 0;
73       else
74          v = p2v[(v2p[v]+a.v2p[a.v]) % gfN];
75       return *this;
76    }
77    GFNUM2m& operator/=(const GFNUM2m &a) {
78       if(a.v == 0) {
79          gferror("Division by zero in Galois field arithmetic\n");
80       }
81       else {
82          if(v == 0) return *this;
83          int x = v2p[v] - v2p[a.v];
84          if(x < 0) x += gfN;
85          v = p2v[x % gfN];
86       }
87       return *this;
88    }
89    GFNUM2m& operator^=(const int exp) {
90       if(v==0) {
91          if(exp == 0) {          //  make 0^0 = 1
92             v = 1;
93             return *this;
94          }
95          else {
96             return *this;
97          }
98       }
99       if(exp < 0) {
100         v = p2v[gfN - ((v2p[v]*(-exp)) % gfN)];
```

```
101         }
102         else {
103             v = p2v[(v2p[v]*exp)% gfN];
104         }
105         return *this;
106     }
107      // unary -
108     GFNUM2m operator-() const { return *this; }
109 
110     GFNUM2m operator+(const GFNUM2m &b) const {return GFNUM2m(v^b.v)
        ;}
111     GFNUM2m operator-(const GFNUM2m &b) const {return GFNUM2m(v^b.v)
        ;}
112     GFNUM2m operator*(const GFNUM2m &b) const {
113         GFNUM2m temp;
114         if(v == 0 || b.v == 0) return temp;
115         temp.v = p2v[(v2p[v] + v2p[b.v]) % gfN];
116         return temp;
117     };
118     GFNUM2m operator/(const GFNUM2m &b) const {
119         if(b.v == 0) {
120             gferror("Division by zero in Galois field arithmetic\n");
121             return 0;
122         }
123         else {
124             GFNUM2m temp;
125             if(v == 0) return temp;
126             int x = v2p[v] - v2p[b.v];
127             if(x < 0) x += gfN;
128             temp.v = p2v[x % gfN];
129             return temp;
130         }
131     }
132 
133     // exponentiation operator: watch the precedence!!
134     // const int e is changed to int e because there is no
135     // way the variable location the user supplies for e is
136     // going to be modified.  The function is more efficient
137     // if it can modify its own copy of e.
138     GFNUM2m operator^(const int e) const {
139         GFNUM2m temp;
140         if(v == 0) {
141             if(e == 0) {  // make 0^0 = 1
142                 temp.v = 1;
143             }
144             return temp;
145         }
146         if(v == 1) {
147             temp.v = 1;
148             return temp;
149         }
```

```
150         if(e < 0) {
151            temp.v = p2v[gfN- ((v2p[v]*(-e)) % gfN)];
152         }
153         else {
154            temp.v = p2v[v2p[v]*e % gfN];
155         }
156         return temp;
157      }
158
159      int operator==(const GFNUM2m &b) const {return (v == b.v);}
160      int operator!=(const GFNUM2m &b) const {return (v != b.v);}
161
162      friend ostream& operator<<(ostream &os, const GFNUM2m &a);
163   };
164
165   inline GFNUM2m operator+(const int a, const GFNUM2m b) { return b+a
         ;}
166   inline GFNUM2m operator-(const int a, const GFNUM2m b) { return b+a
         ;}
167   inline GFNUM2m operator*(const int a, const GFNUM2m b) { return b*a
         ;}
168   inline GFNUM2m operator/(const int a, const GFNUM2m b) { return
         GFNUM2m(a)/b;}
169   inline GFNUM2m operator==(const int a, const GFNUM2m b) { return b==
         a;}
170   inline GFNUM2m operator!=(const int a, const GFNUM2m b) { return b!=
         a;}
171
172   #endif
173
174   /*
175   Local Variables:
176   compile-command: "g++ -c -g GFNUM2m.cc"
177   End:
178   */
```

Listing 1: GFNUM2m.h

```
1   // GFNUM2m.cc --- Function definitions for GFNU2m,
2   // performing Galois field arithmetic
3   // Copyright 2004 by Todd K. Moon
4   // Permission is granted to use this program/data
5   // for educational/research only
6
7   #include "GFNUM2m.h"
8   #include "BinLFSR.h"
9
10  // Define static variables in GFNUM2m
11  int *GFNUM2m::p2v = 0;          // convert exponent to vector
12  int *GFNUM2m::v2p = 0;          // a list of elements to convert
       from
```

```
13                                  // vector to exponential notation
14 int GFNUM2m::gfm = 0;             // vector size of field element
15 int GFNUM2m::gfN = 0;             // number of nonzero-elements in
       field
16 outformat GFNUM2m::outtype = GFpower;
17                                  // default to exponential output
18 // ALPHA elements from the field
19 GFNUM2m ALPHA;                   // define the element alpha
20 GFNUM2m& A= ALPHA;               // and a shorthand reference to it
21
22
23 void GFNUM2m::initgf(int m)
24 {
25    //do the initialization by using only the size of the field
26    // m in GF(2^m).
27    // A fixed set of primitive polynomials is used.
28    // Octal:
29    unsigned int g[] = {1,1,7,013, 023, 045, 0103, 0211, 0435, 01021,
          02011,
30                        04005, 010123, 020033, 042103, 0100003};
31    if(m>sizeof(g)/sizeof(unsigned int)) {
32       cerr << "Error: must specify connection polynomial for m" <<
             endl;
33    }
34    GFNUM2m::initgf(m,g[m]);
35 }
36
37 // initgf: (1) Build the v2p and p2v tables
38 //         (2) set the global variable ALPHA
39 //         (3) set the static member variables gfm and gfN
40 void GFNUM2m::initgf(int m,unsigned int g)
41 // m -- GF(2^m)
42 // g -- generator polynomial, bits represent the coefficients:
43 // e.g. g = 0x13 = 1 0011 = D^4 + D + 1
44 {
45    int i,j;
46
47
48    if(m > sizeof(unsigned int)*8) { // too many bits!
49       cerr << "Error: Degree too large in GFNUM2m" << endl;
50    }
51
52    ALPHA.v = 2;                  // set up alpha element
53    gfm = m;
54    gfN = (1<<m)-1;               // gfN = number of nonzero field
          elements,
55                                  // gfN = 2^n -1
56
57    if(v2p) delete[] v2p;         // delete any prior stuff
58    if(p2v) delete[] p2v;
59
```

```
60      v2p = new int[gfN+1];         // table to convert vector to power
           form
61      p2v = new int[gfN+1];         // tabel to convert power to vector
           form
62
63       BinLFSR X(g, m, 1);
64       v2p[0] = -1;p2v[0] = 1;
65       v2p[1] = 0; p2v[1] = 2;
66       for(int i = 1; i < gfN; i++)
67       {
68           X.step();
69           v2p[X.get_state()] = i;
70           p2v[i] = X.get_state();
71           //cout << "state=" << X.get_state() << " i=" << i+1 << endl;
72       }
73       /*
74       for(int i = 0; i < gfN+1; i++)
75       {
76           //cout << "v2p[" << i << "]=" << v2p[i] << endl;
77           //cout << "p2v[" << i << "]=" << p2v[i] << endl;
78       }*/
79
80  }
81
82
83  ostream& operator<<(ostream& s,const GFNUM2m& arg)
84  {
85      if(arg.getouttype() == GFpower) {
86          if(arg.v < 2) return s << arg.v;
87          else {
88              int e = arg.v2p[arg.v];
89              if(e == 1) return s << "A";
90              else return s << "A^" << e;
91          }
92      }
93      else {                        // vector (numeric) output
94          return s << arg.v;
95      }
96  }
97
98
99  /*
100 Local Variables:
101 compile-command: "g++ -c -g GFNUM2m.cc"
102 End:
103 */
```

Listing 2: GFNUM2m.cc

```
1   //  Program: testgfnum.cc
2   //  test basic operations on GFNUM2m class
```

```
3
4  // Copyright 2004 by Todd K. Moon
5  // Permission is granted to use this program/data
6  // for educational/research only
7
8  #include "GFNUM2m.h"
9
10 int main()
11 {
12    int g = 0x13;  // 1 0011 = 1+x+x^4
13    GFNUM2m::initgf(4,g);
14    GFNUM2m a,b,c;
15    GF2 a2,b2,c2;
16    // GFNUM2m::setouttype(vector);
17    a = A^3;
18    cout << a << endl;
19    b = A^5;
20    cout << b << endl;
21    cout << "a+b=" << a+b << endl;
22    cout << "a+=b=" << (a+= b) << endl;
23    a = A^3;
24    cout << "a+A^9=" << a+10 << endl;
25    cout << "a+A^9=" << 10+a << endl;
26    cout << "a=" << a << "  b=" << b << endl;
27    cout << "a*b=" << a*b << endl;
28    cout << "a*=b=" << (a*=b) << endl;
29    a = A^3;
30    cout << "a*8=" << (a*8) << endl;
31    cout << "a*8=" << (8*a) << endl;
32    cout << "a*=8=" << (a*=8) << endl;
33    a = A^3;
34    cout << "a*A^5=" << a*(A^5) << endl;
35    cout << "a^4=" << (a^4) << endl;
36    cout << "a^{-4}=" << (a^(-4)) << endl;
37    cout << "a^=4=" << (a^=4) << endl;
38    a = A^3;
39    cout << "a^=(-4)=" << (a^=(-4)) << endl;
40    a = A^3;
41    cout << "a/b=" << a/b << endl;
42    cout << "a/=b=" << (a/=b) << endl;
43    a = A^3;
44    cout << "a/6=" << a/6 << endl;
45    cout << "a/=6=" << (a/=6) << endl;
46    a = A^3;
47    cout << "1/a=" <<  (1/a) << endl;
48    a = A^3;
49    cout << "a==b" << (a==b) << endl;
50    cout << "a!=b" << (a!=b) << endl;
51    cout << "-a=" << -a << endl;
52    a2 = 1;
53    b = a+a2;
```

```
54      cout << "a2=" << a2 << "  b=" << b << endl;
55  }
56
57  /*
58  Local Variables:
59  compile-command: "g++ -g -o testgfnum testgfnum.cc GFNUM2m.cc
        BinLFSR.cc"
60  End:
61  */
```

Listing 3: testgfnum$_4$.*cc*

```
1   //  Program: testgfnum.cc
2   //  test basic operations on GFNUM2m class
3
4   // Copyright 2004 by Todd K. Moon
5   // Permission is granted to use this program/data
6   // for educational/research only
7
8   #include "GFNUM2m.h"
9
10  int main()
11  {
12      int g = 0x11D;  // 1 0011 = 1+x+x^4
13      GFNUM2m::initgf(8,g);
14      GFNUM2m a,b,c;
15      GF2 a2,b2,c2;
16      // GFNUM2m::setouttype(vector);
17      a = A^3;
18      cout << a << endl;
19      b = A^5;
20      cout << b << endl;
21      cout << "a+b=" << a+b << endl;
22      cout << "a+=b=" << (a+= b) << endl;
23      a = A^3;
24      cout << "a+A^9=" << a+10 << endl;
25      cout << "a+A^9=" << 10+a << endl;
26      cout << "a=" << a << "  b=" << b << endl;
27      cout << "a*b=" << a*b << endl;
28      cout << "a*=b=" << (a*=b) << endl;
29      a = A^3;
30      cout << "a*8=" << (a*8) << endl;
31      cout << "a*8=" << (8*a) << endl;
32      cout << "a*=8=" << (a*=8) << endl;
33      a = A^3;
34      cout << "a*A^5=" << a*(A^5) << endl;
35      cout << "a^4=" << (a^4) << endl;
36      cout << "a^{-4}=" << (a^(-4)) << endl;
37      cout << "a^=4=" << (a^=4) << endl;
38      a = A^3;
39      cout << "a^=(-4)=" << (a^=(-4)) << endl;
```

```
40      a = A^3;
41      cout << "a/b=" << a/b << endl;
42      cout << "a/=b=" << (a/=b) << endl;
43      a = A^3;
44      cout << "a/6=" << a/6 << endl;
45      cout << "a/=6=" << (a/=6) << endl;
46      a = A^3;
47      cout << "1/a=" <<  (1/a) << endl;
48      a = A^3;
49      cout << "a==b" << (a==b) << endl;
50      cout << "a!=b" << (a!=b) << endl;
51      cout << "-a=" << -a << endl;
52      a2 = 1;
53      b = a+a2;
54      cout << "a2=" << a2 << "  b=" << b << endl;
55  }
56
57  /*
58  Local Variables:
59  compile-command: "g++ -g -o testgfnum testgfnum.cc GFNUM2m.cc
        BinLFSR.cc"
60  End:
61  */
```

Listing 4: testgfnum$_8$.*cc*